UPEN

Career: Engineering in Information Technologies and Digital Innovation

Subject: Object-Oriented Programming

Activity: Justification of the class diagram

Student: Alex Antonio Pinto Cataño

Delivery Date: September 14, 2026

1. The problem and the choice of diagram

The system models the structure of a soccer tournament or league. It requires storing information about the people involved (players, coaching staff, referees), team

composition, match schedules at their respective stadiums, and recording key events during the game (goals and cards). A class diagram was chosen because, unlike a sequence diagram that shows the flow of a process over time, this diagram defines the static structure of the data. It answers the question: "What entities make up the system, what information do they store, and how are they connected?"

2. Participants and Key Relationships

- **Person (Superclass):** This is the base class. It is used to avoid redundancy of common attributes (Name, BirthDay, BirthPlace) in other entities.
- **Player / CoachingStaff / Referee:** They inherit from Person but add data specific to their role in the tournament (Position, Role, Federation).
- **Team:** Groups the players and coaching staff to represent a country.
- **Match and Stadium:** A match centralizes the event, associating teams and referees, and physically takes place in a stadium.
- **Goal and Card: These are** game events that have their own state (the minute they occur) and must be related to the player who executes or receives them.

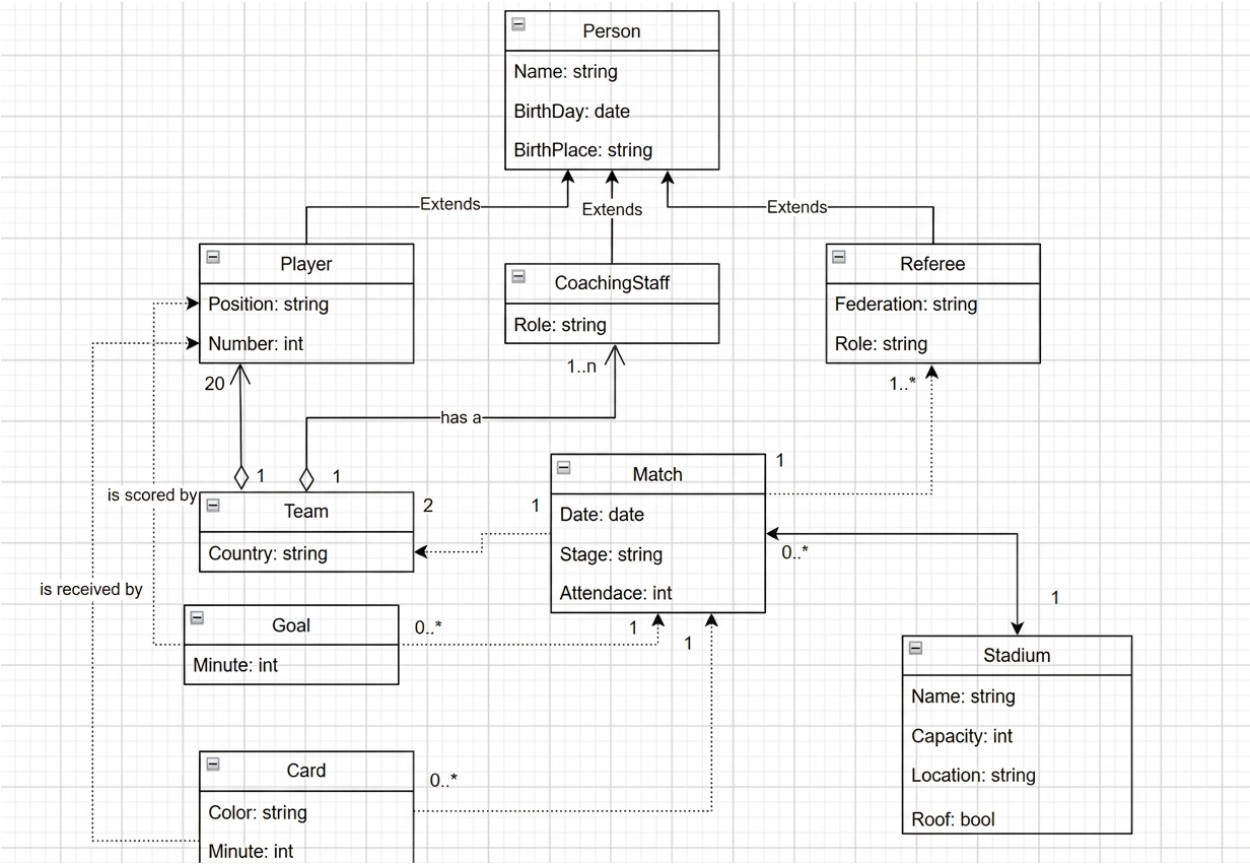
Relationship	Justification (why it exists)	Notation used
Inheritance (Person)	Code reuse; everyone is a person before their specific role.	Solid arrow with a hollow point ("Extends").
Aggregation (Team)	A team consists of exactly 20 players and 1 or more members of the coaching staff.	Hollow rhombus next to the Team class.
Match	It is a necessary logical consequence: a match always requires exactly 2 teams to take place.	Solid line with multiplicity 2.

3. Covered Cases and Notation

- **Cases it does cover: The** hierarchical organization of people, the strict formation of team rosters, the allocation of venues (stadiums) and the traceability of who scores goals or receives cards.
- **Cases it does NOT cover (and why that's okay):** Dynamic actions or step-by-step flows (like the process of registering a new player), as that would correspond to a sequence diagram. It also doesn't model a standings table, since points can be logically calculated from the results of each match.
- **Notation reading:** The rectangles represent the classes or objects of the system. The numbers and ranges (1, 0..*, 20) on the lines indicate the multiplicity, that is, how many instances of a class are allowed in that relation.

4. Possible questions from the teacher

- **Why did you separate Goal and Card instead of putting them as attributes in Match?**
- A: Because they are independent events that have their own attributes (like the Minute) and are linked to a specific Player (is scored by / is received by), not just the match in general.
- **Why is the hollow rhombus on Team's side?**
- A: Because in UML that is the standard notation for aggregation, indicating that the team is the "whole" that groups the "parts" (the players and the coaching staff).



Conclusion

Creating this diagram was like putting together a puzzle where the main challenge was organizing a lot of information without repeating data and keeping everything logically connected:

- **Avoid repeating information:** Initially, the challenge was realizing that players, coaching staff members, and referees share many things.basic data. Instead of

writing the name and date of birth in each one, the best decision was to create a general category ("Person") to group that information shared only once.

- **Organizing the center of it** all: A football match involves many different elements. The design challenge was figuring out how to precisely group two teams, assign referees, and establish a physical stadium within a single event. We succeeded in making the "Match" function as the central point where all these independent parts intersect in an orderly fashion.

- **Recording precise game details:** It seemed simple enough to just record goals and cards as a number within the match, but that created a problem: we needed to *know* who scored the goal and in what *minute*. Therefore, the solution was to separate goals and cards into their own categories, allowing us to directly link them to the specific player who performed the action or received the penalty.

- **Knowing where to stop:** Finally, we had to decide what not to include in the diagram to avoid overcomplicating it. We agreed to leave out step-by-step actions (like the dynamic process for registering a new player) or things that can be deduced later (like the tournament standings). This helped us keep the focus strictly on how the league data is structured and stored.